ÉTUDE DE CAS — STRIPE

LIVRABLE 0

Contexte & Objectifs

Certification RNCP 41993 – Adrien COSSON
Bloc 2 - Concevoir et déployer l'infrastructure de données et de calcul pour l'IA

1. Contexte

Stripe est une entreprise leader dans les technologies financières, qui propose des solutions de paiement et de gestion financière pour des entreprises de différentes tailles. L'entreprise traite chaque année un très grand nombre de transactions et doit donc gérer des volumes de données importants.

Avec sa croissance, Stripe fait face à une complexité de plus en plus importante dans la gestion de ses données, qui sont réparties sur différents types de systèmes, notamment des systèmes transactionnels (OLTP), analytiques (OLAP) et des bases de données NoSQL.

Ces données sont de natures très différentes. On retrouve d'un côté les données transactionnelles (paiements, remboursements, rétrofacturations, abonnements), qui sont structurées, et de l'autre des données non structurées ou semi-structurées comme les logs, les données de navigation sur le site et les applications (clickstream), les avis clients ou encore les features utilisées par les modèles de Machine Learning. À cela s'ajoutent des données de référence utilisées par tous les systèmes (pays, devises, taux de change, informations sur les marchands).

Le principal enjeu pour Stripe est donc de pouvoir gérer efficacement cette diversité de données et de systèmes, tout en continuant à accompagner la croissance de l'entreprise.

Pour bien répondre aux exigences réglementaires liées aux données de paiement ainsi qu'aux contraintes de fonctionnement, Stripe doit répondre aux exigences suivantes :

- **La sécurité :** Stripe doit répondre à la norme de sécurité des données de carte bancaire (PCI-DSS).
- **L'aspect réglementaire :** chaque pays est régi par ses propres réglementations, comme le RGPD pour l'Union Européenne ou la CCPA pour la Californie. Stripe doit donc être en mesure de respecter ces différentes réglementations en fonction du pays dans lequel son service est déployé.
- **La rapidité :** au moment du paiement, la transaction doit être vérifiée (notamment par le modèle de détection de fraude) puis acceptée ou refusée en moins de 100 ms. Le client final ne doit pas ressentir d'attente, sinon le marchand risque de perdre la vente.
- **La fiabilité du système et de son architecture :** l'architecture de Stripe doit être adaptée à l'échelle de ses besoins. Les données issues des systèmes OLTP, OLAP et NoSQL doivent pouvoir être intégrées efficacement afin de rester cohérentes, à jour et accessibles pour les différents usages. Cela permet également d'éviter les doublons dans les bases de données ou encore plusieurs prélèvements pour une même transaction.
- **La disponibilité :** le système de paiement ne doit jamais être arrêté. Il faut donc prévoir un système de secours qui prend le relais en cas de panne (failover) et un plan de reprise après un incident majeur (disaster recovery).
- **La maîtrise des coûts :** la direction financière souhaite un coût mensuel prévisible pour l'infrastructure. Les choix techniques doivent donc aussi être justifiés d'un point de vue économique.

2. Objectifs

L'objectif de ce dossier est de proposer une architecture de données complète pour Stripe qui permettra d'intégrer les différents systèmes de données utilisés par l'entreprise : OLTP ; OLAP et NoSQL.

L'architecture proposée doit répondre aux besoins de Stripe, en particulier en matière de traitement des transactions, l'analyse des données, ainsi que la gestion des données non-structurées et semi-structurées. Enfin, elle doit aussi permettre d'assurer la cohérence et la synchronisation des données entre les différents systèmes.

L'architecture devra également répondre aux contraintes liées aux performances, à la scalabilité, à la sécurité et à la conformité des données financières. Une attention particulière sera portée à la capacité de l'architecture à gérer les flux de données en temps réel.

Enfin, le dossier devra montrer comment cette architecture peut permettre d'intégrer des cas d'usage liés au Machine Learning, notamment pour la détection de fraude et la personnalisation client.

3. Parties prenantes

Plusieurs acteurs sont concernés par cette architecture, à l'intérieur comme à l'extérieur de Stripe, et ils n'ont pas tous les mêmes attentes :

Acteur	Qui ?	Ce qu'il attend
Marchands	Les entreprises qui utilisent Stripe pour encaisser leurs ventes	Que les paiements de leurs clients soient acceptés rapidement et sans erreur, et que les fraudes soient bloquées.
Clients finaux	Les personnes qui paient sur le site d'un marchand	Payer vite et en sécurité, et garder le contrôle sur leurs données personnelles.
Équipes paiement	Elles développent et maintiennent le système de paiement	Un système toujours disponible qui enregistre chaque transaction correctement.
Équipe fraude	Elle analyse les paiements suspects	Notification réactive et fiabilité des alertes, sans être noyée sous les fausses alertes.
Data scientists	Ils créent, évaluent, améliorent et suivent les modèles de Machine Learning	Avoir tout l'historique pour entraîner les modèles et les données récentes pour les utiliser en temps réel sur des données de qualité.
Analystes	Ils suivent les revenus, les clients et les produits	Lancer leurs requêtes rapidement, même sur de gros volumes et obtenir de l'information fiable.
Conformité et DPO	Ils vérifient que Stripe respecte le RGPD, la CCPA et PCI-DSS ou toute autre réglementation en fonction du pays	Savoir qui a accédé à quelles données et pouvoir sortir facilement les rapports de conformité.
Direction (CTO, CFO)	Elle valide les choix techniques et le budget	Une architecture capable de suivre la croissance, avec une anticipation des coûts mensuels.

4. Exigences

Le tableau ci-dessous reprend, pour chaque partie de l'architecture, ce qu'elle doit permettre de faire (exigence fonctionnelle) et le niveau attendu (exigence non fonctionnelle), ainsi que le livrable dans lequel ce point sera traité. Les deux dernières lignes concernent l'architecture dans son ensemble.

(*) Ces valeurs ne sont pas données dans le sujet. Ce sont des hypothèses que j'ai fixées pour pouvoir dimensionner l'architecture dans les livrables suivants.

Composant	Exigence fonctionnelle	Exigence non fonctionnelle	Livrable
Transactions (OLTP)	Enregistrer tous les paiements, remboursements, rétrofacturations et abonnements en respectant les propriétés ACID.	La base doit répondre avec une latence minimale et rester disponible 99,99 % du temps (*). En cas de panne, une base de secours prend le relais et aucune transaction ne doit être perdue.	L2 · L1
Détection de fraude	Donner un score de fraude à chaque transaction avant de l'accepter et prévenir l'équipe fraude en cas de doute.	Le score doit être calculé en moins de 100 ms, même pendant les pics de trafic. Le modèle utilise le comportement récent du client, mis à jour en continu, et son historique.	L7 · L1
Pipeline de données	Faire circuler les données entre OLTP, OLAP et NoSQL en batch et en streaming. Le CDC (Change Data Capture) permet de récupérer les changements sans ralentir la base transactionnelle.	Quasi temps réel (quelques secondes) pour le streaming (*) et toutes les heures pour le chargement du data warehouse (*), sans perte ni doublon. Entre les systèmes, une cohérence à terme suffit.	L5 · L1
Données non structurées (NoSQL)	Stocker les logs, le clickstream, les avis clients et les features ML, et pouvoir les relier aux autres systèmes.	Le schéma doit rester flexible (JSON, XML, fichiers binaires) et les données imbriquées doivent pouvoir être interrogées rapidement grâce aux index.	L4
Analyse (OLAP)	Analyser les revenus, la segmentation client, les performances des produits et la conformité, et répondre aux questions métier avec des requêtes SQL et NoSQL.	Les requêtes doivent rester rapides sur de gros volumes, d'où l'utilisation de tables pré-agrégées et de vues matérialisées. Les données récentes doivent être disponibles en moins d'une heure (*).	L3 · L8

Composant	Exigence fonctionnelle	Exigence non fonctionnelle	Livrable
Machine Learning	Faire de l'analyse prédictive, puis suivre et mettre à jour les modèles.	Score de fraude calculé en temps réel. Les performances des modèles sont suivies et les modèles mis à jour lors d'un réentraînement (chaque semaine, ou plus tôt en cas de dérive).	L7
Sécurité et conformité	Garder une trace des accès et des opérations, et permettre à une personne de consulter ou de supprimer ses données.	Accès limités selon le rôle de chacun, respect du RGPD, de la CCPA et de PCI-DSS. Les failles doivent être détectées en temps réel (*) et des audits de conformité réalisés régulièrement.	L6 · L2
Toute l'architecture	–	Scalabilité : supporter des millions de transactions par jour (10 millions en moyenne et 1 200 par seconde en pic (*)) en ajoutant des machines (scalabilité horizontale).	L1 · L3 · L4 · L5
Toute l'architecture	–	Disponibilité : chaque composant est réparti sur plusieurs centres de données, avec un plan de reprise dans une seconde région en cas de sinistre.	L1 · L2
Infrastructure de calcul	Fournir la puissance de calcul (CPU, GPU) et le stockage nécessaires à l'entraînement et à l'inférence des modèles.	Ressources dimensionnées selon les volumes attendus, ajustées automatiquement à la charge. Les machines d'entraînement sont arrêtées après usage.	L1 · L7
Toute l'architecture	–	Coûts : avoir un coût mensuel prévisible, identifier les plus gros postes de dépense et proposer une solution pour réduire chacun d'eux.	L1

5. Bénéfices clés

La mise en place de cette architecture doit apporter plusieurs bénéfices à Stripe :

- **Fiabilité des transactions :** garantir l'intégrité et la cohérence des données lors du traitement des paiements, remboursements et autres opérations financières, sans doublon ni opération manquante.
- **Meilleure exploitation des données :** permettre de centraliser et d'exploiter les données provenant des différents systèmes afin de faciliter les analyses et la prise de décision.
- **Performance et scalabilité :** pouvoir traiter des volumes de données importants tout en conservant de bonnes performances et permettre à l'architecture d'évoluer avec la croissance de Stripe. Cela permettra d'améliorer la résilience de Stripe.
- **Données disponibles en temps réel :** permettre la circulation des données entre les différents systèmes afin de répondre à des besoins nécessitant une information récente, notamment pour la détection de fraude. L'architecture devient ainsi réactive.
- **Sécurité et conformité :** protéger les données sensibles et répondre aux différentes exigences réglementaires applicables aux données financières (RGPD, CCPA, PCI-DSS).
- **Support des cas d'usage Machine Learning :** faciliter l'utilisation des données pour développer des modèles de détection de fraude, de personnalisation et d'analyse prédictive. Les points décrits précédemment permettront aussi une bonne exécution des modèles grâce à une meilleure traçabilité des données.
- **Maîtrise des coûts :** s'appuyer sur des services managés et sur l'autoscaling pour ne payer que les ressources réellement utilisées, et donner à la direction financière une vision claire du coût de l'infrastructure.

6. Périmètre du dossier

Le dossier couvre la conception de l'architecture de données de Stripe et les différents composants nécessaires à son fonctionnement. Il comprend les éléments suivants :

- **Architecture globale des données :** Présentation de l'architecture cible et des interactions entre les systèmes OLTP, OLAP et NoSQL, ainsi que les différents flux de données. Présentation de

l'infrastructure (hébergement, architecture physique, dimensionnement, haute disponibilité, déploiement) et estimation des coûts. **[Livrable 1]**

- **Modèle de données OLTP :** Conception d'un modèle de données relationnel normalisé permettant de gérer les transactions financières et de garantir leur intégrité. **[Livrable 2]**
- **Modèle de données OLAP :** Conception d'un modèle adapté à l'analyse des données, avec notamment un schéma en étoile, ainsi que des mécanismes permettant d'améliorer les performances des requêtes. **[Livrable 3]**
- **Modèle de données NoSQL :** Conception d'un modèle permettant de gérer les données non structurées et semi-structurées, avec une réflexion sur les relations entre les documents et les stratégies d'indexation. **[Livrable 4]**
- **Pipeline d'intégration des données :** Présentation des flux permettant de synchroniser les différents systèmes, avec la prise en compte des traitements batch et temps réel, ainsi que des mécanismes ETL/ELT et CDC. **[Livrable 5]**
- **Sécurité et conformité :** Définition des principales mesures de sécurité concernant le chiffrement, les contrôles d'accès et la journalisation, ainsi que les mesures permettant de répondre aux exigences réglementaires comme le RGPD pour l'Union Européenne et le PCI-DSS (internationale). **[Livrable 6]**
- **Intégration du Machine Learning :** Présentation de la manière dont les données peuvent être utilisées pour alimenter des modèles de Machine Learning, notamment pour la détection de fraude et la personnalisation client. **[Livrable 7]**
- **Requêtes SQL et NoSQL :** Mise en place de plusieurs requêtes permettant d'illustrer l'utilisation des modèles proposés pour répondre à des besoins métier comme l'analyse des revenus, la segmentation client ou la détection de fraude. **[Livrable 8]**